

REPEATED SIGNALS

Build the receiver. Know when to stop.



An illustrated C# + WPF build guide

Captain Vale needs a clear count while Ashwake clears the Training Ring. You are the systems runner. Build a small receiver model from a plain window; count packets, recognize STOP and enforce a six-packet limit.

Actions 4A + 4B build most of the same weekly application. Add your runner identity, test it, then submit one accessible GitHub repository link through Canvas.

Read silently or use the optional scene voices on the website. Code is selectable text. The text-only HTML guide contains the same directions.

The escape / story transcript



The corridor is closing.

Captain Vale: Hold on. That cargo frame is coming across our bow. The receiver is repeating the same distress call, and I need a clear count before we choose a route.

Ashwake banks beneath a tumbling cargo frame. The courier recorder is aboard; the disputed warrant remains unresolved.



Do not let the noise decide.

Captain Vale: Runner, take the receiver. Read a small batch, count the packets, and stop when the stop marker arrives. If the marker never comes, six packets is the limit. Keep the rest for review.

You are Ashwake's systems runner. You can build and test the receiver model; you cannot declare a distress call genuine or clear a disputed warrant.



Make the stopping rule visible.

Captain Vale: Start with a plain window. Build the controls, connect the click, then trace the loop. Ask for help with one small change. Run it. Tell me what it actually did.

Your artifact is a C# WPF receiver in Visual Studio. The tutorial teaches a finite classroom batch, not live networking or flight controls.



A clear count. A way through.

Captain Vale: Good. We have room to move. A clear count does not make the message true. Save the working project and your tests. Bring back the repository link, and keep the unanswered questions with it.

Ashwake clears the immediate corridor. The source of the repeating message remains unknown. Continue the Quill investigation without inventing an answer.

Your route through Week 4

READ

In your purchased *Attention First* by Malik A. Stalbert, revisit Chapter 4, Your SEEK Story Framework, and Chapter 2, Designing Your AAA Sequence, already used in Week 3. Use the section titles if page numbers differ. In your README, write 3-4 sentences: what your runner needs to notice, one interface choice that directs attention, and how a test changed your next action. Apply the reading to this app; no separate worksheet or book-page upload.

WATCH / FOLLOW

Read or listen to the four illustrated story beats. Work through tutorial 1-7 below. The PDF and website tutorial are equivalent: do not repeat both as separate tasks.

COMPLETE

Build the WPF receiver in Visual Studio. Action 4A covers steps 1-3. Action 4B advances the same work in steps 4-6. Step 7 prepares the weekly submission.

SUBMIT

One tested GitHub repository URL in the official CS077 Week 4 Canvas assignment. Include project source, used assets, README, screenshot, class planning/collaboration evidence and tests.

Access and scope

Use the Windows/.NET desktop tools already used in Week 3. Required AI has a supplied-response route with no personal or paid account. If the book, software, GitHub or required collaboration route is inaccessible, preserve your work and contact the instructor through Canvas or course Discord. The illustrated software is a classroom model, not a live receiver.

The raw starter has no completed interface, loop or reset. Each tutorial step tells you where to edit and what you should see. Keep the working core before attempting the AI addition.

01 / Open a genuinely raw project.

4A / CS077.RepeatedSignals.csproj



1. Download the raw starter ZIP. Right-click it in File Explorer and choose Extract All. Put the extracted folder in your local CS077/Week04 folder; do not run a project inside the ZIP.
2. Open Visual Studio (not Visual Studio Code). Choose Open a project or solution and select CS077.RepeatedSignals.csproj. If prompted for a workload, use Visual Studio Installer > Modify > .NET desktop development. The starter targets .NET 8 for Windows, matching Week 3; ask the instructor before changing the target.
3. In View > Solution Explorer, find MainWindow.xaml and expand it to find MainWindow.xaml.cs. Press F5. A plain window saying Start here is the expected result. Close that window before editing.
4. Choose Git > Create Git Repository. Name it CS077-Week04-RepeatedSignals and use your existing GitHub account. Review the local path, leave secrets and personal files out, then Create and Push. If a repository already exists here, use Git Changes instead of nesting another repo.

Why this works

WPF builds a Windows desktop interface. XAML describes controls and layout; the .xaml.cs code-behind describes behavior. The .csproj file tells Visual Studio how to build the project. A Git commit records a local checkpoint; Push sends commits to GitHub.

WPF: Windows Presentation Foundation: a desktop UI framework.

XAML: Markup that declares controls and their properties.

Code-behind: The C# file linked to the XAML window.

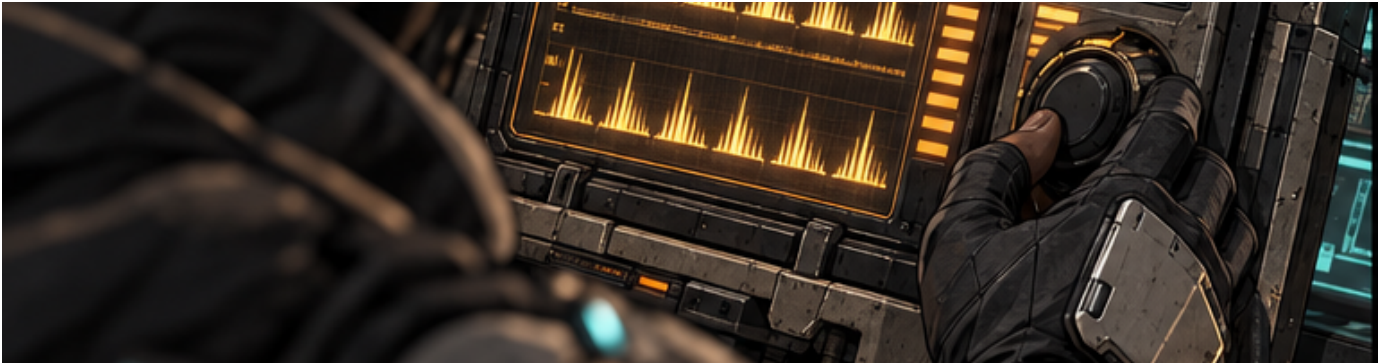
Repository: A project history stored locally and, after pushing, on GitHub.

Check / recovery

The raw project opens and runs. No Scan button or receiver logic exists yet. In GitHub, confirm the project files are visible. If GitHub is blocked, keep a local commit and contact the instructor; do not substitute an inaccessible link.

02 / Build the receiver's controls.

4A / MainWindow.xaml



1. Stop debugging. Open MainWindow.xaml and switch to the XAML text view.
2. Select the entire contents of MainWindow.xaml and replace them with the example below. Do not paste into App.xaml or MainWindow.xaml.cs. Keep x:Class exactly as shown for this starter.
3. Read the controls from top to bottom: heading, input, button, output. Change only the heading to include your Week 3 runner's callsign. Keep the x:Name values unchanged so the next code can find them.
4. Save and press F5. You should see an input containing five packet labels, a Scan batch button, and Ready. The button intentionally does nothing until the next step.

Why this works

StackPanel lays out controls in order. TextBox accepts the batch; TextBlock displays output. x:Name gives C# a way to refer to a control. MaxLength keeps this classroom input small. The ScrollViewer keeps the window usable at smaller sizes.

Property: A setting such as Text, Margin or FontSize.

x:Name: The identifier used to find a XAML control from C#.

Event: Something that happens, such as clicking a button.

Step 2 code / MainWindow.xaml

Use the exact paste directions in this step. Read the code as you add it.

```
<Window x:Class="CS077.RepeatedSignals.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Title="Ashwake | Repeated Signals" Height="520" Width="760"
  MinHeight="420" MinWidth="440">
  <ScrollViewer VerticalScrollBarVisibility="Auto">
    <StackPanel Margin="24">
      <TextBlock Text="Ashwake receiver" FontSize="28"/>
      <TextBlock Text="Packets (separate with semicolons)" Margin="0,16,0,4"/>
      <TextBox x:Name="PacketInput" MaxLength="200"
        Text="PING;PING;DISTRESS;STOP;PING"/>
      <Button x:Name="ScanButton" Content="Scan batch"
        HorizontalAlignment="Left" Padding="18,8" Margin="0,12"/>
      <TextBlock x:Name="SignalOutput" Text="Ready."
        TextWrapping="Wrap" FontFamily="Consolas"/>
    </StackPanel>
  </ScrollViewer>
</Window>
```

Check / recovery

All controls are visible. A normal click does nothing yet. If the XAML fails, check quotes, closing tags and the unchanged x:Class. Undo only your latest edit, not the whole project.

03 / Connect the click and count.

4A / MainWindow.xaml.cs + one XAML attribute



1. Stop debugging. Open MainWindow.xaml.cs and replace that file's entire contents with the example. This is the only step that replaces the full code-behind file.
2. Return to MainWindow.xaml. On the Button element, add Click="ScanButton_Click" after x:Name="ScanButton". Save both files.
3. Press F5, then Scan batch. At this stage every packet is counted, including STOP: the default input reports Processed: 5. STOP is just text until you teach the app its meaning in Action 4B.
4. On the whiteboard, trace `i` and processed for the first three passes. With a classmate, explain why `i` starts at zero and `processed++` adds one. Share a cropped running-app picture and the whiteboard photo in the course Discord. Save a local commit and push.

Why this works

Split turns semicolon-separated text into an array (an ordered group). `packets.Length` is its size; `packets[i]` reads one element. `for` starts `i` at zero, checks its condition before each pass, and increments `i` after the pass. `ProcessPackets` is a method returning a string; the event handler puts that string on screen. `\n` starts a new line.

Array / index: An ordered group / a position starting at zero.

Counter: A number updated to track how many items were processed.

Iteration: One pass through the loop body.

Return: Sends the method's result back to its caller.

Step 3 code / MainWindow.xaml.cs + one XAML attribute

Use the exact paste directions in this step. Read the code as you add it.

```
using System;
using System.Windows;

namespace CS077.RepeatedSignals;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void ScanButton_Click(object sender, RoutedEventArgs e)
    {
        string[] packets = PacketInput.Text.Split(';',
            StringSplitOptions.RemoveEmptyEntries |
            StringSplitOptions.TrimEntries);
        SignalOutput.Text = ProcessPackets(packets);
    }

    public static string ProcessPackets(string[] packets)
    {
        int processed = 0;
        string log = "";
        for (int i = 0; i < packets.Length; i++)
        {
            log += $"{i + 1}. {packets[i]}\n";
            processed++;
        }
        return log + $"Processed: {processed}";
    }
}
```

Check / recovery

Default input produces Processed: 5; PING;DISTRESS produces 2; empty input produces 0. If the button does nothing, check its Click attribute. If PacketInput is unknown, check the exact x:Name in XAML.

04 / Teach the receiver when to stop.

4B / MainWindow.xaml.cs > ProcessPackets only



1. Begin Action 4B by reopening the same project. Recap the 4A trace with a classmate. Explain why STOP was counted yesterday.
2. Replace only the complete ProcessPackets method, from its signature through its matching closing brace, with the example below. Leave the using lines, namespace, class, constructor and ScanButton_Click unchanged.
3. Read the two bounds in the for condition. The app must have another packet AND the processed count must be below six. Inside the loop, check STOP before logging or increasing the count.
4. Run the five tests in the next lesson. In the debugger, set a breakpoint on `if (packets[i] == "STOP")`, click Scan batch and use F10 to step. Watch `i`, `packets[i]` and `processed` in Locals. Remove the breakpoint when finished.

Why this works

STOP is a sentinel: a special value that ends processing. `break` exits this loop; it does not close the app. `&&` requires both conditions to be true. The fixed limit also stops a batch when STOP is absent. Work is limited to six small items; do not add `while(true)`, `Thread.Sleep`, or a real network listener on the window's UI thread.

Sentinel: A reserved value used to end a sequence.

`break`: Immediately exits the nearest loop.

Boundary test: Checks behavior at a limit, including zero and the maximum.

Breakpoint: Pauses debugging at a chosen code line.

Step 4 code / MainWindow.xaml.cs > ProcessPackets only

Use the exact paste directions in this step. Read the code as you add it.

```
public static string ProcessPackets(string[] packets)
{
    int processed = 0;
    string log = "";
    for (int i = 0; i < packets.Length && processed < 6; i++)
    {
        if (packets[i] == "STOP")
        {
            log += "STOP received.\n";
            break;
        }
        log += $"{processed + 1}. {packets[i]}\n";
        processed++;
    }
    if (processed == 6)
    {
        log += "Safety limit reached.\n";
    }
    return log + $"Processed: {processed}";
}
```

Check / recovery

PING;PING;DISTRESS;STOP;PING now reports 3. STOP is not counted and the later PING is not read. A six-item cap keeps this teaching batch bounded; it is not a general solution for slow live-network tasks.

05 / Prove the stopping rules.

4B / README.md + Evidence/



1. Run every test below in Visual Studio. Replace the input each time; click Scan batch once. Expected results are supplied. Record your actual result separately.
2. Repeat the default scan twice. Each click must report 3, not 6: processed is local and starts at zero for each call.
3. Capture one readable screenshot showing your receiver and a result. Save the whiteboard loop trace and a short Discord peer-check record in Evidence. Include only course-related content; crop unrelated names or messages.
4. In README, add your Attention First connection and the tests. If something differs, record the exact result or error and fix the core behavior before the final weekly submission.

Why this works

A passing ordinary case does not prove the edges work. Empty input, STOP first, a missing STOP and an oversized batch each test a different stopping path. Comparing expected and observed results makes debugging specific.

Expected: What the specification says should happen.

Observed: What actually happened when you ran your code.

Input	Expected	Observed
PING;PING;DISTRESS;STOP;PING	3 packets. STOP received; the last PING is ignored.	Record your result.
STOP;PING	0 packets. STOP first; no data packet processed.	Record your result.
PING;DISTRESS;PING	3 packets. End of batch; no STOP needed.	Record your result.
(empty)	0 packets. Empty batch; zero loop passes.	Record your result.
A;B;C;D;E;F;G;H	6 packets. Safety limit; G and H are not processed.	Record your result.

Check / recovery

The five required cases pass and a repeated click does not accumulate counts. The browser simulator is only a practice model; screenshots and results must come from your own WPF app.

06 / Ask AI for one small change.

4B / MainWindow.xaml + MainWindow.xaml.cs



1. Commit the working core before this extension. The required AI task is to add a Reset receiver button that restores the default batch and Ready. It does not change the scanning algorithm.
2. Copy the supplied prompt. Add only your two relevant code files, with no passwords, tokens or private information. Use the class-approved AI tool; if live access is unavailable, study and test the supplied sample response below instead.
3. Ask where each piece belongs. Add the button beside ScanButton inside StackPanel. Add its handler inside MainWindow, outside the other methods. Run it and check both the input and output.
4. In your README record the prompt or supplied-response route, which files/lines you changed, your test and observed result, and what you kept, changed, rejected or will test next. Share the current state with your class partner. Do not claim it works without running it.

Why this works

This is Mindset Lab: Search, Test, Continue, guided by your instructor. An unfamiliar task is an invitation to ask a precise question and test a small change. A genuine unfinished reset attempt plus its error/result and next test meets this process requirement. Keep the core receiver working; save an unfinished experiment separately or in a clearly labeled Git branch.

Baseline: A known-working version you can return to.

Verification: Checking the generated suggestion against actual behavior.

Copy this prompt

I am a beginner using C# and WPF in Visual Studio with the CS077.RepeatedSignals starter. My core receiver already works. Help me add ONE Reset receiver button. It must restore PacketInput.Text to "PING;PING;DISTRESS;STOP;PING" and SignalOutput.Text to "Ready." Keep ProcessPackets unchanged. Show the smallest XAML addition and C# handler, the exact file and insertion location for each, and explain Click, x:Name, and the handler signature. Give me one test with its expected result. Do not replace my whole project. Here are my MainWindow.xaml and MainWindow.xaml.cs files: [paste only these files, with no private information].

Supplied response / account-free route

Add this button after ScanButton inside StackPanel in MainWindow.xaml:

```
<Button Content="Reset receiver" Click="ResetButton_Click"
  HorizontalAlignment="Left" Padding="18,8" Margin="0,0,0,12"/>
```

Add this handler inside MainWindow in MainWindow.xaml.cs, outside its other methods:

```
private void ResetButton_Click(object sender, RoutedEventArgs e)
{
    PacketInput.Text = "PING;PING;DISTRESS;STOP;PING";
    SignalOutput.Text = "Ready.";
}
```

Click names the handler. sender identifies the source control; RoutedEventArgs carries event information; void returns no value. The handler changes the two controls named in XAML. Run a scan, change the input, click Reset receiver, then scan again. Expect the default input, Ready., and then Processed: 3. Record that you used the supplied response; test it rather than assuming it works.

Check / recovery

A successful reset restores PING;PING;DISTRESS;STOP;PING and Ready. If unfinished, preserve the attempted code and complete inquiry note; a description without an attempt is not enough.

07 / Make it yours. Save and submit.

Week 4 / GitHub repository > Canvas



1. Add one small original detail connected to your Week 3 runner: a callsign heading, readable palette, short mission message or original portrait with alt/context text. Keep the required control names and stopping rules. Explain the choice in one sentence.
2. In Visual Studio > Git Changes, review the changed files. Include the .csproj, App.xaml/.cs, MainWindow.xaml/.cs, any used assets, README and Evidence. Do not commit bin, obj, .vs, credentials or unrelated private material.
3. Write a meaningful commit message. Choose Commit All, then Push (or Commit All and Push). Visit GitHub and confirm the newest commit and every required file. A local commit alone is not online.
4. Test access as the grader: for public repos, open the URL signed out; for private repos, give the instructor access through the approved class route. Do not expose private information to make a link public.
5. Open the official CS077 Week 4 Canvas assignment. Submit the tested repository URL in Website URL, then reopen your submission. The portal's checklist, export, Discord post and Git push do not submit to Canvas.

Why this works

Your final artifact is the same application built in Actions 4A and 4B, refined into a readable repository. Git keeps local history and an online copy. Canvas records the official submission. Dates and accepted settings come from Canvas.

Commit: Save a version in local Git history.

Push: Send local commits to the remote repository.

Submission: The separate act of returning the link through Canvas.

Check / recovery

The repository contains source and evidence, the instructor can open it, and Canvas shows the submitted link. If access is blocked, preserve local work and contact the instructor before the close.

Completion requirements

R1 * Apply Attention First

Revisit the assigned SEEK and AAA sections. In README, connect a runner need, an interface choice and a test-driven revision in 3-4 sentences.

R2 * Build the WPF receiver

Start raw. Add the labeled input, Scan batch button and readable output. Include your runner's callsign and one small original detail.

R3 * Count and stop correctly

Use a C# loop and counter. STOP ends the scan without being counted. Never process more than six packets per click.

R4 * Show all five tests

Record expected and observed results for the five supplied cases, plus the repeated-click check. Include a readable screenshot of your running WPF app.

R5 * Attempt the AI reset task

Use the prompt or supplied response. Save the actual code attempt, changed files, result/error and next test in README. The reset may be unfinished; the core receiver must work.

R6 Keep the class evidence

Complete Actions 4A and 4B in class with instructor-verified attendance and participation. Include each meeting's whiteboard trace and concise Discord peer-check record. Reuse class evidence; do not create duplicate reports.

R7 * Submit the repository link

Commit and push source, assets, README and Evidence. Test instructor access, submit the GitHub URL to the Week 4 Canvas assignment, and reopen it.

Return to Canvas

Attendance and grading

Attendance and participation are required. Both Action 4A and Action 4B must be completed and accepted by the instructor before Week 4 is accepted for grading. An upload or personal checkmark does not verify attendance. If you miss a meeting, contact the instructor about any approved arrangement under the syllabus and applicable accommodations. Once accepted for grading, all R1-R7 are required; a missing starred requirement reduces the weekly record to one point. R5 requires a genuine attempt and complete inquiry record, not a working reset. R2-R4 require a working core receiver.

One weekly submission: your accessible GitHub repository URL. Include your project source and the evidence named in R1-R7. An exported guide, Discord post or browser checkmark is not a Canvas submission.

Action Records

Actions are completed with the instructor in the scheduled class meeting. Each requires whiteboard/shared planning, course collaboration, the meeting build and instructor verification. An upload alone does not establish attendance or earn the Action Point. Reuse these files in your weekly repository.

If you are stuck

Save your work locally first. Capture the exact error or access issue. Contact the instructor through Canvas or course Discord before the announced close. Do not delete a working baseline while experimenting with the AI addition.

References and help

[C# loops](#)

[WPF project and controls](#)

[Git in Visual Studio](#)

[CS077 Canvas](#)

Source documentation checked September 12, 2026. Original teaching examples and narrative: CS077 course package. Attention First is required course reading and is not reproduced in this guide.